CUESTIONARIO 10

Colecciones y genéricos

Pregunta 1

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

In the Java collections framework a Set Is:

Seleccione una:

a. An ordered collection that can contain duplicate elements.

b. A collection that cannot contain duplicate elements. ✓

c. An object that maps value key sets and cannot contain values Duplicates.

Justificación: Un Set es una colección que no admite elementos duplicados, según equals. La opción a describe una List y la c describe, con mala redacción, un Map.

Fuente: Java E #17 · también aparece en: Java Sesiones #37

La respuesta correcta es: A collection that cannot contain duplicate elements.

Mi justificación
Un set no puede tener elementos duplicados por definición.

Pregunta 2

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Cuál de las siguientes afirmaciones sobre ArrayList es verdadera?

Seleccione una:

- a. ArrayList permite elementos duplicados.
- b. ArrayList no mantiene el orden de inserción de los elementos.
- c. Los elementos de ArrayList se almacenan en una estructura de árbol.
- d. ArrayList puede almacenar tipos primitivos directamente.

Justificación: ArrayList es una lista respaldada por un arreglo dinámico: permite duplicados y mantiene el orden de inserción. No usa una estructura de árbol ni almacena primitivos directamente: guarda wrappers como Integer gracias al autoboxing.

Fuente: Simulación APX V1 #43 · también aparece en: Java.pdf #58

La respuesta correcta es: ArrayList permite elementos duplicados.

Mi justificación
Un ArrayList permite elementos duplicados, el que no permite es Set.

Pregunta 3

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Cuál es la característica que comparten Array y Arraylist?

Seleccione una:

a. Pueden almacenar valores nulos.
b. Ambos procesos ocurren en tiempo constante.
c. Es posible que tengan valores duplicados. ✓
d. Array y ArrayList no garantizan la presencia de elementos ordenados.

Justificación: Tanto un arreglo como una ArrayList pueden contener valores repetidos. Lo de los null no siempre se cumple: un arreglo de primitivos (int[]) no admite null. Ambos mantienen el orden por índice, así que la d es falsa, y no todas sus operaciones son de tiempo constante (por ejemplo, insertar en medio de una ArrayList).

Fuente: Java.pdf #148 · también aparece en: Simulación APX V2 #42, Cuestionario APX V3 #8

La respuesta correcta es: Es posible que tengan valores duplicados.

Mi justificación
Array y ArrayList permiten elementos duplicados. Array no permite nulos.

Pregunta 4

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Cuál de las siguientes no es parte del framework de Java Collections?

Seleccione una:

- a. ArrayList
- b. Map
- c. Dictionary ✓
- d. List

Justificación: Dictionary es una clase abstracta heredada de Java 1.0 (padre de Hashtable), anterior al Java Collections Framework y ajena a él. Map es el distractor tentador porque no extiende Collection, pero sí forma parte del framework.

Fuente: Simulación APX V1 #12 · también aparece en: Java.pdf #23

La respuesta correcta es: Dictionary

Mi justificación
Collections tiene a ArrayList, Map y List, al que no tiene es a Dictionary ya que son pares de valores y no una colección como tal.

Pregunta 5

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Qué método en java.lang.Object debe ser sobrescrito si se sobrescribe el método hashCode()?

Seleccione una:

- a. clone()
- b. compareTo()
- c. copy()
- d. equals() ✓
- e. sort()

Justificación: hashCode y equals van juntos: dos objetos iguales según equals deben tener el mismo hashCode, porque HashMap y HashSet dependen de ello. compareTo es de Comparable, no de Object. copy y sort no existen en Object, y clone no está ligado a hashCode.

Fuente: Simulación APX V2 #35 · también aparece en: Java.pdf #130

La respuesta correcta es: equals()

?

Mi justificación
El hashCode determina si dos objetos son iguales por el hashCode, pero podemos cambiar este criterio por algún atributo de la clase, así que sobreescribirlo modifica el método equals().

Pregunta 6

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

What is the output?

```
import java.util.ArrayList;
import java.util.List;
public class Test {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        list.add("Apple");
        list.add("Banana");
        list.add("Cherry");
        printList(list);
        System.out.println("Size: " + list.size());
        System.out.println("Contains 'Banana': " + list.contains("Banana"));
        System.out.println("Element at index 1: " + list.get(1));
    }
    private static void printList(List<String> list) {
        for (String element : list) {
            System.out.println(element);
        }
    }
}
```

Seleccione una:

- a. Apple Banana Cherry Size: 3 Contains 'Banana': true Element at index 1: Banana ✓
- b. Apple Banana Size: 2 Contains 'Banana': false Element at index 1: Banana
- c. Apple Banana Size: 2 Contains 'Banana': true Element at index 1: Banana
- d. Apple Banana Cherry Size: 3 Contains 'Banana': true Element at index 1: Cherry

Justificación: La lista tiene 3 elementos en orden de inserción. contains("Banana") devuelve true y, como los índices empiezan en 0, get(1) es "Banana". La d supone que los índices empiezan en 1. (Cada println va en su propia línea; las opciones la muestran en una sola.)

Fuente: Java.pdf #97 · también aparece en: Simulación APX V4 #32

La respuesta correcta es: Apple Banana Cherry Size: 3 Contains 'Banana': true Element at index 1: Banana

?

Mi justificación
Bastante directo. Se imprime la lista completa: Apple, Banana, Cherry Se imprime el tamaño: 3 Se imprime si contiene Banana: true. Se imprime el elemento en el índice 1, el cual es Banana. Es la a)

Pregunta 7
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Nivel: **Básico**

Dado el siguiente código, ¿cuál es el resultado?

```
import java.util.*;

public class Ordenamiento {
    public static void main(String[] args) {
        List<Integer> numeros = new ArrayList<Integer>();
        numeros.add(8);
        numeros.add(5);
        numeros.add(3);
        numeros.add(6);

        Collections.sort(numeros);
        System.out.println(numeros);
    }
}
```

Seleccione una:

- a. [8, 5, 3, 6]
- b. [3, 5, 6, 8] ✓
- c. Compilation error
- d. [8, 6, 5, 3]

Justificación: Collections.sort(lista) ordena según el orden natural de Integer, que es ascendente, y println de una List imprime los elementos entre corchetes: [3, 5, 6, 8] (verificado con JDK 21). La a es el orden de inserción, que se tendría sin ordenar. La d es el orden descendente, que requeriría Collections.reverseOrder(). Con el import java.util.* el código compila, así que la c no aplica.

Nota sobre el material original: El código del material no incluía «import java.util.*;» y, tal cual, no compila (cannot find symbol: List, ArrayList, Collections). Se añadió el import porque la intención de la pregunta es evaluar el ordenamiento. Además, las opciones con listas se escribieron entre corchetes, que es como las imprime println.

Fuente: Quiz Desarrollo APX 2 #14

La respuesta correcta es: [3, 5, 6, 8]

Mi justificación
El método sort() ordena la lista en orden ascendente así que queda 3,5,6,8.

Pregunta 8

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Cuál de las siguientes implementaciones de código utiliza correctamente una clase genérica en Java?

Seleccione una o más de una:

a.

```
public class Box {
    private Object value;
    public Box(Object value) {
        this.value = value;
    }
    public Object getValue() {
        return value;
    }
}
```

b.

```
public class Box {
    private int value;
    public Box(int value) {
        this.value = value;
    }
    public int getValue() {
        return value;
    }
}
```

c.

```
public class Box<T> {
    private T value;
    public Box(T value) {
        this.value = value;
    }
    public T getValue() {
        return value;
    }
}
```

d.

```
public class Box<T extends Number> {
    private T value;
    public Box(T value) {
        this.value = value;
    }
    public T getValue() {
        return value;
    }
}
```

Justificación: Una clase genérica declara un parámetro de tipo (<T>) y lo usa en sus campos y métodos. La c lo hace sin restricciones y la d con un límite superior (T extends Number): las dos usan genéricos correctamente. La a usa Object, sin genéricos y con necesidad de casts, y la b usa un int fijo. Ninguna de esas dos es una clase genérica, aunque compilen.

Fuente: Cuestionario APX V3 #2 - también aparece en: Java.pdf #164

Las respuestas correctas son:

```
public class Box<T> {
    private T value;
    public Box(T value) {
        this.value = value;
    }
    public T getValue() {
        return value;
    }
}
```

```
public class Box<T extends Number> {
    private T value;
    public Box(T value) {
        this.value = value;
    }
    public T getValue() {
        return value;
    }
}
```

Mi justificación
Las que tienen el tipo T en sus constructores y parámetro de Box son las correctas ya que T es el genérico.

Pregunta 9

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

¿Qué clase puede proveer las siguientes capacidades?: no permitir valores duplicados, mantener el orden en el cual los elementos fueron insertados, proveer iteración rápida y acceso aleatorio eficiente.

Seleccione una:

a. TreeSet
b. LinkedHashSet ✓
c. HashSet
d. ArrayList
e. LinkedHashMap

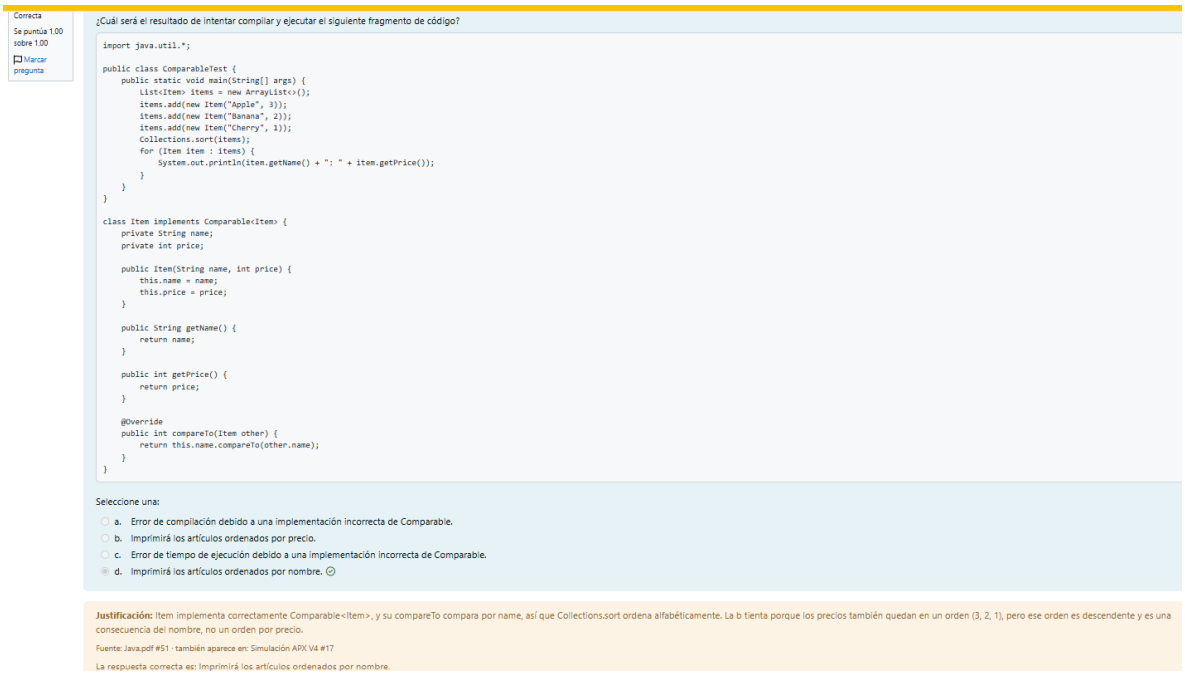
Justificación: LinkedHashSet no admite duplicados, conserva el orden de inserción e itera rápido, así que es la única clase que cumple las condiciones principales. HashSet no garantiza ningún orden, TreeSet ordena por valor y ArrayList admite duplicados. Ninguna clase de la lista ofrece además acceso por índice sin duplicados: ese requisito del enunciado es impreciso.

Fuente: Simulación APX V2 #38 · también aparece en: Java.pdf #134

La respuesta correcta es: LinkedHashSet

?

Mi justificación
Para no permitir duplicados usamos un Set y para añadir que queremos mantener el orden de inserción usamos el LinkedHashSet.



Correcta

Se puntúa 1.00 sobre 1.00

Marcar pregunta

¿Cuál será el resultado de intentar compilar y ejecutar el siguiente fragmento de código?

```
import java.util.*;

public class ComparableTest {
    public static void main(String[] args) {
        List<Item> items = new ArrayList<>();
        items.add(new Item("Apple", 3));
        items.add(new Item("Banana", 2));
        items.add(new Item("Cherry", 1));
        Collections.sort(items);
        for (Item item : items) {
            System.out.println(item.getName() + ": " + item.getPrice());
        }
    }
}

class Item implements Comparable<Item> {
    private String name;
    private int price;

    public Item(String name, int price) {
        this.name = name;
        this.price = price;
    }

    public String getName() {
        return name;
    }

    public int getPrice() {
        return price;
    }

    @Override
    public int compareTo(Item other) {
        return this.name.compareTo(other.name);
    }
}
```

Seleccione una:

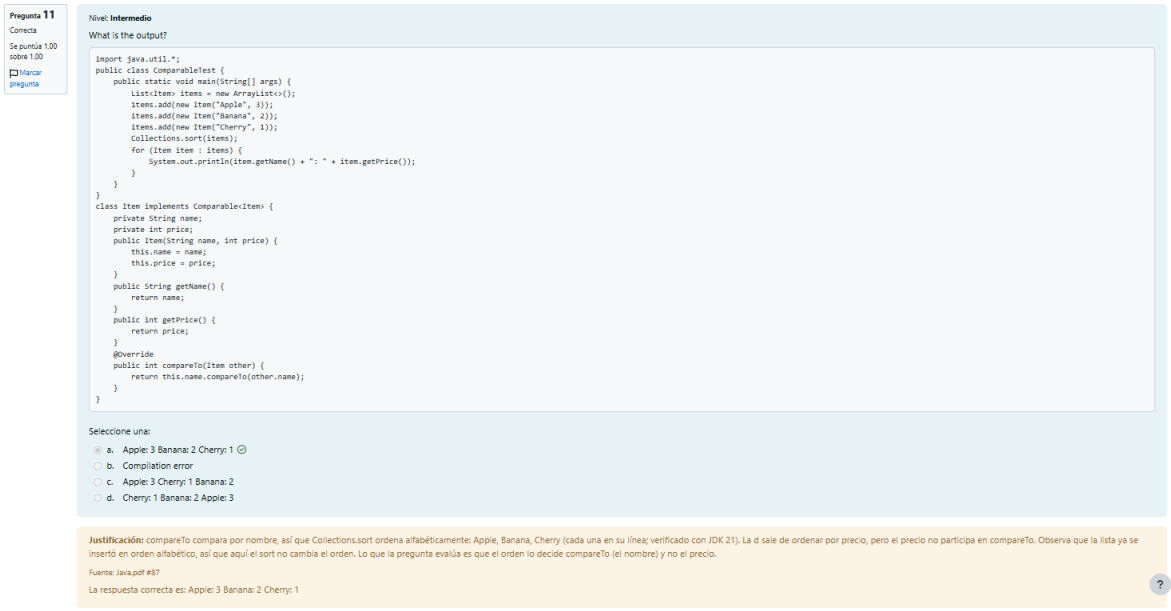
a. Error de compilación debido a una implementación incorrecta de Comparable.
b. Imprimirá los artículos ordenados por precio.
c. Error de tiempo de ejecución debido a una implementación incorrecta de Comparable.
d. Imprimirá los artículos ordenados por nombre.

Justificación: Item implementa correctamente Comparable<Item>, y su compareTo compara por name, así que Collections.sort ordena alfabéticamente. La b tienta porque los precios también quedan en un orden (3, 2, 1), pero ese orden es descendente y es una consecuencia del nombre, no un orden por precio.

Fuente: Java.pdf #51 · también aparece en: Simulación APX V4 #17

La respuesta correcta es: Imprimirá los artículos ordenados por nombre.

Mi justificación
La interfaz Comparable se implementa y se sobreescribe el método compareTo con return this.name.compareTo(other.name). Esta notación hace que al usar compareTo sobre varios objetos, el método sort() los ordene por orden alfabético.



Pregunta 11

Correcta

Se puntúa 1.00 sobre 1.00

Marcar pregunta

Nivel: **Intermedio**

What is the output?

```
import java.util.*;
public class ComparableTest {
    public static void main(String[] args) {
        List<Item> items = new ArrayList<>();
        items.add(new Item("Apple", 3));
        items.add(new Item("Banana", 2));
        items.add(new Item("Cherry", 1));
        Collections.sort(items);
        for (Item item : items) {
            System.out.println(item.getName() + ": " + item.getPrice());
        }
    }
}
class Item implements Comparable<Item> {
    private String name;
    private int price;
    public Item(String name, int price) {
        this.name = name;
        this.price = price;
    }
    public String getName() {
        return name;
    }
    public int getPrice() {
        return price;
    }
    @Override
    public int compareTo(Item other) {
        return this.name.compareTo(other.name);
    }
}
```

Seleccione una:

a. Apple: 3 Banana: 2 Cherry: 1

b. Compilation error

c. Apple: 3 Cherry: 1 Banana: 2

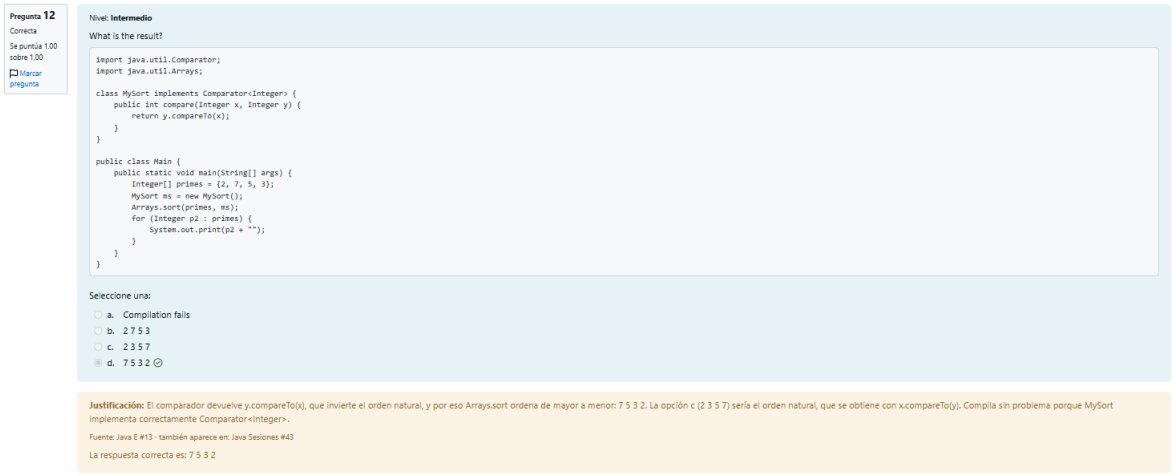
d. Cherry: 1 Banana: 2 Apple: 3

Justificación: compareTo compara por nombre, así que Collections.sort ordena alfabéticamente: Apple, Banana, Cherry (cada una en su línea; verificado con JDK 21). La d sale de ordenar por precio, pero el precio no participa en compareTo. Observa que la lista ya se insertó en orden alfabético, así que aquí el sort no cambia el orden. Lo que la pregunta evalúa es que el orden lo decide compareTo (el nombre) y no el precio.

Fuente: Java.pdf #87

La respuesta correcta es: Apple: 3 Banana: 2 Cherry: 1

Mi justificación
La interfaz Comparable se implementa y se sobreescribe el método compareTo con return this.name.compareTo(other.name). Esta notación hace que al usar compareTo sobre varios objetos, el método sort() los ordene por orden alfabético.



Pregunta 12
Correcta
Se puntúa 1.00 sobre 1.00
Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
import java.util.Comparator;
import java.util.Arrays;

class MySort implements Comparator<Integer> {
    public int compare(Integer x, Integer y) {
        return y.compareTo(x);
    }
}

public class Main {
    public static void main(String[] args) {
        Integer[] primes = {2, 7, 5, 3};
        MySort ms = new MySort();
        Arrays.sort(primes, ms);
        for (Integer p2 : primes) {
            System.out.print(p2 + "");
        }
    }
}
```

Seleccione una:

a. Compilation fails
b. 2 7 5 3
c. 2 3 5 7
d. 7 5 3 2

Justificación: El comparador devuelve y.compareTo(x), que invierte el orden natural, y por eso Arrays.sort ordena de mayor a menor: 7 5 3 2. La opción c (2 3 5 7) sería el orden natural, que se obtiene con x.compareTo(y). Compila sin problema porque MySort implementa correctamente Comparator<Integer>.

Fuente: Java E #13 · también aparece en: Java Sesiones #43

La respuesta correcta es: 7 5 3 2

Mi justificación
Implementar compare(x,y) como return y.compareTo(x) hace que la colección quede en orden inverso al usar sort(), entonces la lista se imprime como 7,5,3,2.

Pregunta 13

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
import java.util.Arrays;
import java.util.Comparator;
class SortLongs implements Comparator<Long> {
    @Override
    public int compare(Long a, Long b) {
        return b.compareTo(a);
    }
}
public class Test {
    public static void main(String[] args) {
        Long[] values = {20L, 70L, 50L, 30L};
        SortLongs sort = new SortLongs();
        Arrays.sort(values, sort);
        System.out.println(Arrays.toString(values));
    }
}
```

Seleccione una:

a. An ArrayIndexOutOfBoundsException is thrown.
b. [20, 70, 50, 30]
c. [20, 30, 50, 70]
d. [70, 50, 30, 20] ✓
e. Compilation fails.

Justificación: El comparador devuelve b.compareTo(a), que invierte el orden natural, así que Arrays.sort deja el arreglo de mayor a menor: [70, 50, 30, 20]. [20, 30, 50, 70] sería el orden natural (a.compareTo(b)). No hay ninguna excepción porque no se accede a índices fuera de rango.

Fuente: Java.pdf #152 · también aparece en: Cuestionario APX V3 #1

La respuesta correcta es: [70, 50, 30, 20]

?

Mi justificación
Implementar compare(a,b) como return b.compareTo(a) hace que la colección quede en orden inverso al usar sort(), entonces la lista se imprime como 70, 50, 30, 20.

Pregunta 14

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

Dado el siguiente código. ¿Cuál es el resultado?

```
public class Ordenamiento {
    public static void main(String[] args) {
        Integer[] valores = {4, 8, 1, 6};
        Arrays.sort(valores, Collections.reverseOrder());

        for (Integer v : valores) {
            System.out.println(v);
        }
    }
}
```

Seleccione una:

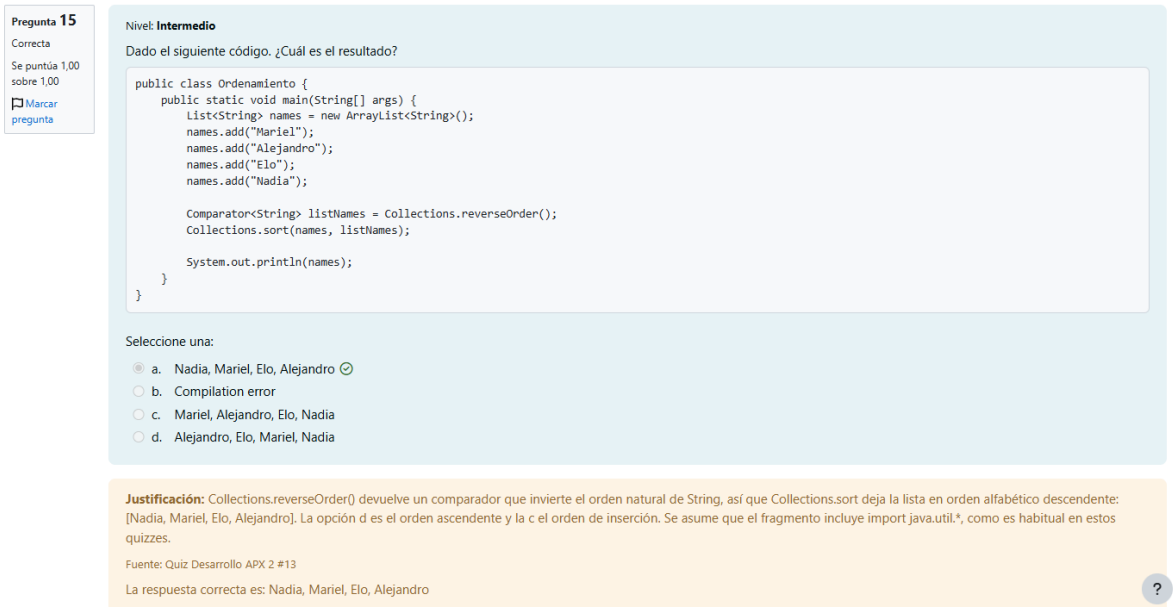
a. 4,8,1,6
b. RuntimeException
c. 8,6,4,1
d. 1,4,6,8

Justificación: Collections.reverseOrder() sirve con Arrays.sort porque el arreglo es de Integer (objetos), y lo ordena de forma descendente: 8, 6, 4, 1. Con un int[] primitivo el código ni siquiera compilaría, así que nunca llegaría a lanzar una RuntimeException en ejecución. Se asume el import de java.util que la captura omite.

Fuente: Quiz Desarrollo APX 2 #15

La respuesta correcta es: 8,6,4,1

Mi justificación
El método reverseOrder() ordena la lista al revés, así que se imprime en orden inverso la lista, quedando 8,6,4,1.



Pregunta 15

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

Dado el siguiente código. ¿Cuál es el resultado?

```
public class Ordenamiento {
    public static void main(String[] args) {
        List<String> names = new ArrayList<String>();
        names.add("Mariel");
        names.add("Alejandro");
        names.add("Elo");
        names.add("Nadia");

        Comparator<String> listNames = Collections.reverseOrder();
        Collections.sort(names, listNames);

        System.out.println(names);
    }
}
```

Seleccione una:

a. Nadia, Mariel, Elo, Alejandro

b. Compilation error

c. Mariel, Alejandro, Elo, Nadia

d. Alejandro, Elo, Mariel, Nadia

Justificación: Collections.reverseOrder() devuelve un comparador que invierte el orden natural de String, así que Collections.sort deja la lista en orden alfabético descendente: [Nadia, Mariel, Elo, Alejandro]. La opción d es el orden ascendente y la c el orden de inserción. Se asume que el fragmento incluye import java.util.*, como es habitual en estos quizzes.

Fuente: Quiz Desarrollo APX 2 #13

La respuesta correcta es: Nadia, Mariel, Elo, Alejandro

Mi justificación
El método reverseOrder() ordena la lista al revés, así que se imprime en orden inverso la lista, quedando Nadia, Mariel, Elo, Alejandro.

Pregunta 16

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

¿Cuáles de los siguientes fragmentos de código implementan correctamente Comparator para ordenar una lista de objetos Person por su edad en orden ascendente? (Elige dos)

```
import java.util.Comparator;

class Person {
    private String name;
    private int age;
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
    public int getAge() {
        return age;
    }
    public String getName() {
        return name;
    }
}
```

Seleccione una o más de una:

a.
```
import java.util.Comparator;
public class AgeComparator implements Comparator<Person> {
    @Override
    public int compare(Person p1, Person p2) {
        return p1.getAge() - p2.getAge();
    }
}
```

b.
```
import java.util.Comparator;
public class AgeComparator implements Comparator<Person> {
    @Override
    public int compare(Person p1, Person p2) {
        return p1.getName().compareTo(p2.getName());
    }
}
```

c.
```
import java.util.Comparator;
public class AgeComparator implements Comparator<Person> {
    @Override
    public int compare(Person p1, Person p2) {
        return Integer.compare(p1.getAge(), p2.getAge());
    }
}
```

c.
```
import java.util.Comparator;
public class AgeComparator implements Comparator<Person> {
    @Override
    public int compare(Person p1, Person p2) {
        return Integer.compare(p1.getAge(), p2.getAge());
    }
}
```

d.
```
import java.util.Comparator;
public class AgeComparator implements Comparator<Person> {
    @Override
    public int compare(Person p1, Person p2) {
        return p2.getAge() - p1.getAge();
    }
}
```

Justificación: a y c ordenan por edad de menor a mayor: la resta p1.getAge() - p2.getAge() e Integer.compare(p1.getAge(), p2.getAge()) devuelven negativo, cero o positivo de forma equivalente. Integer.compare es la forma recomendada, porque la resta puede desbordar con valores extremos. b ordena por nombre, no por edad. d compara edades, pero invierte el orden (de mayor a menor). Las cuatro compilan; con edades 20, 40 y 30, a y c dan 20-30-40 y d da 40-30-20 (verificado con JDK 21).

Nota sobre el material original: El material no trae clave, pregunta «¿Cuál…?» en singular y no dice en qué sentido se ordena, lo que dejaba la d (edad descendente) en duda. Se precisó «en orden ascendente» y «(Elige dos)», porque a y c son equivalentes.

Fuente: Java.pdf #144

Las respuestas correctas son:

```
import java.util.Comparator;
public class AgeComparator implements Comparator<Person> {
    @Override
    public int compare(Person p1, Person p2) {
        return p1.getAge() - p2.getAge();
    }
}
```

```
import java.util.Comparator;
public class AgeComparator implements Comparator<Person> {
    @Override
    public int compare(Person p1, Person p2) {
        return Integer.compare(p1.getAge(), p2.getAge());
    }
}
```

Mi justificación
Para obtener la lista en orden ascendente por edad, debemos usar Person.age y restar los parámetros usados en compareTo(a,b) como a – b o como compare(a,b) Los que cumplen con estas reglas son a y c.

Pregunta 17

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
import java.util.*;
public class App{
    public static void main(String[]args){
        List p = new ArrayList();
        p.add(7);
        p.add(1);
        p.add(5);
        p.add(1);
        p.remove(1);
        System.out.println(p);
    }
}
```

Seleccione una:

- a. [7, 1]
- b. [7, 1, 5, 1]
- c. [7, 5, 1] ✓
- d. [7, 5]

Justificación: Con un literal int, p.remove(1) invoca remove(int index) y elimina el elemento de la posición 1 (el primer 1), no el objeto Integer 1. La lista queda [7, 5, 1]. [7, 5] supondría que se eliminan todas las apariciones del valor 1, cosa que no hace ninguna sobrecarga de remove.

Fuente: Java D #15 · también aparece en: Java Sesiones #1

La respuesta correcta es: [7, 5, 1]

?

Mi justificación
El método remove() solo remueve la primera aparición del parámetro en la lista así que pasa de 7,1,5,1 a 7,5,1.

Pregunta 18

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
public class MyStuff {
    String name;
    MyStuff (String n) { name = n; }
    public static void main (String[] args) {
        MyStuff m1 = new MyStuff ("guitar");
        MyStuff m2 = new MyStuff ("tv");
        System.out.println (m2.equals(m1));
    }
    public boolean equals (Object o) {
        MyStuff m = (MyStuff) o;
        if (m.name != null) { return true; }
        return false;
    }
}
```

Seleccione una:

a. The output is true and MyStuff fulfills the Object.equals() contract

b. The output is false and MyStuff fulfills the Object.equals() contract

c. The output is true and MyStuff does NOT fulfill the Object.equals() contract. ✓

d. The output is false and MyStuff does NOT fulfill the Object.equals() contract

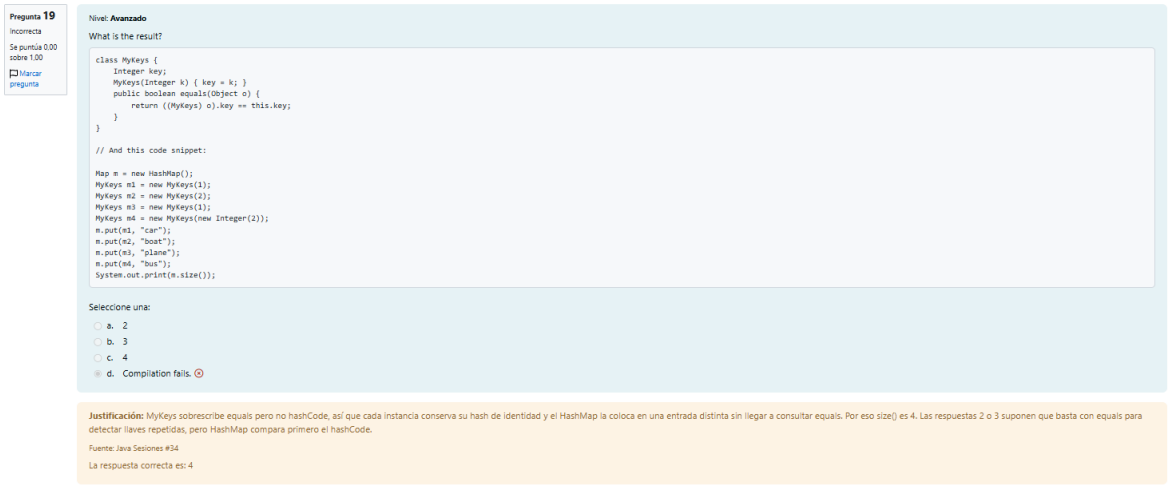
Justificación: m2.equals(m1) devuelve true porque m1.name no es null. El método incumple el contrato de equals: considera iguales objetos distintos ("guitar" y "tv"), lanza NullPointerException con equals(null) en vez de devolver false, lanza ClassCastException con otros tipos y no sobrescribe hashCode. Que la salida sea true no prueba que el contrato se cumpla.

Fuente: Java Sesiones #35 · también aparece en: Java DE #10

La respuesta correcta es: The output is true and MyStuff does NOT fulfill the Object.equals() contract.

?

Mi justificación
Los objetos son distintos, así que no cumple con el contrato de Object.equals(), pero equals está sobreescrito para que regrese true con la condición de que m.name no sea null, y esto se cumple ya que m.name = guitar.



Pregunta **19**

Incorrecta

Se puntúa 0.00 sobre 1.00

Marcar pregunta

Nivel: **Avanzado**

What is the result?

```
class MyKeys {
    Integer key;
    MyKeys(Integer k) { key = k; }
    public boolean equals(Object o) {
        return ((MyKeys) o).key == this.key;
    }
}

// And this code snippet:

Map m = new HashMap();
MyKeys m1 = new MyKeys(1);
MyKeys m2 = new MyKeys(2);
MyKeys m3 = new MyKeys(1);
MyKeys m4 = new MyKeys(new Integer(2));
m.put(m1, "car");
m.put(m2, "boat");
m.put(m3, "plane");
m.put(m4, "bus");
System.out.print(m.size());
```

Seleccione una:

a. 2
b. 3
c. 4
d. Compilation fails.

Justificación: MyKeys sobrescribe equals pero no hashCode, así que cada instancia conserva su hash de identidad y el HashMap la coloca en una entrada distinta sin llegar a consultar equals. Por eso size() es 4. Las respuestas 2 o 3 suponen que basta con equals para detectar llaves repetidas, pero HashMap compara primero el hashCode.

Fuente: Java Sesiones #34

La respuesta correcta es: 4

Mi justificación
No compila ya que m4 = new MyKeys (new Integer(2)) hace que ese la lista quede con tipos de datos mezclados y eso es incompatible así que falla la compilación.

RESULTADO = 18/19 = 9.47